

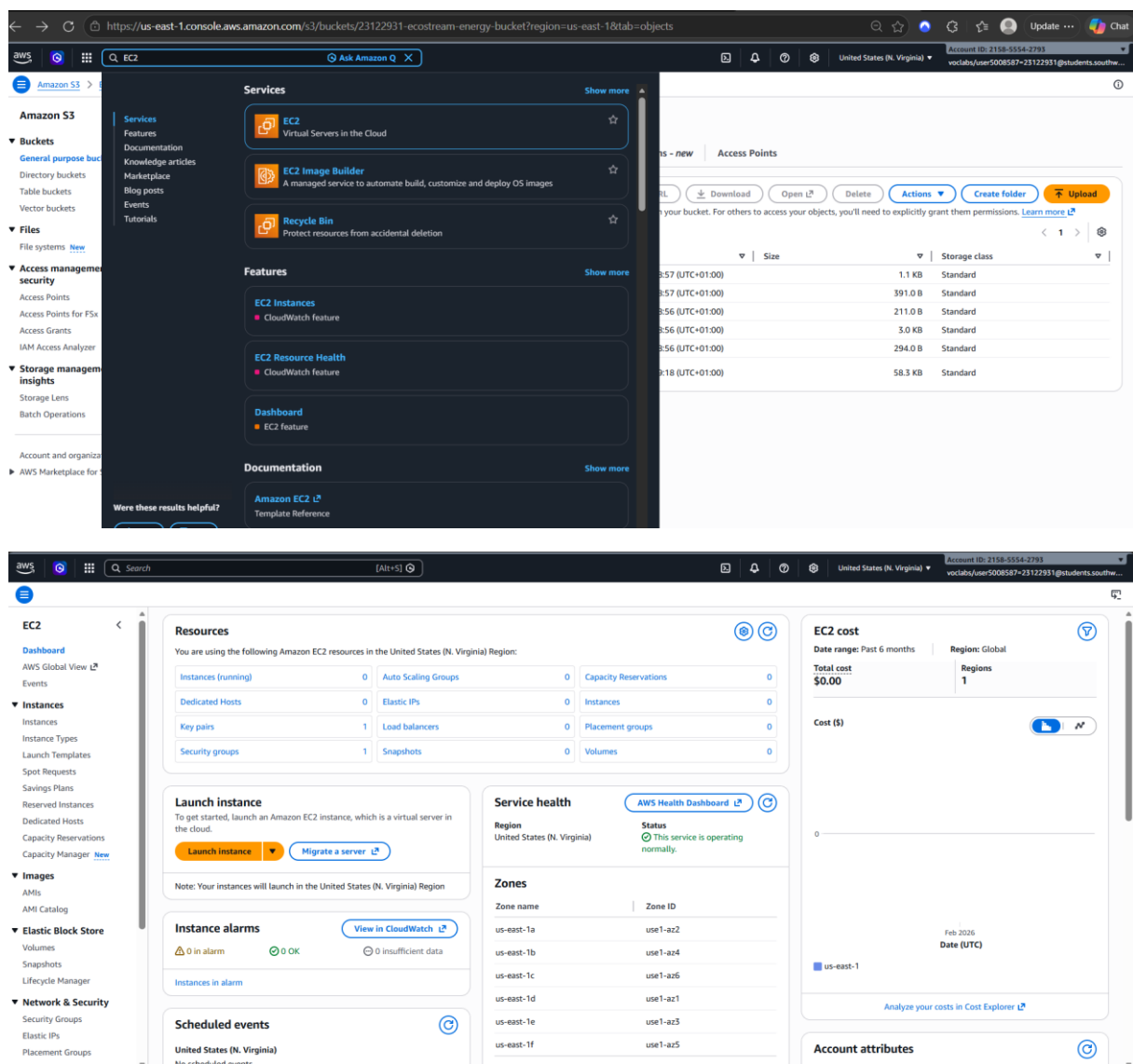
Amazon Machine Image Step-by-Step Guide

Introduction

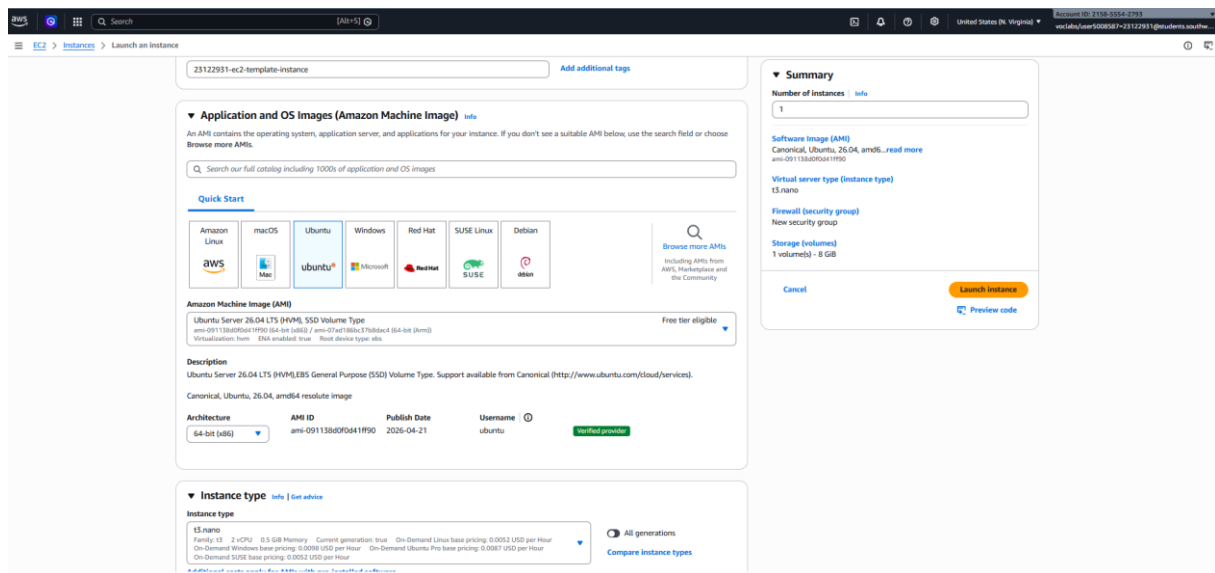
This document presents a step-by-step method for creating and validating a custom Amazon Machine Image for EcoStream Energy using Amazon EC2 services. The guide explains how a typical Ubuntu Linux EC2 instance was set up as a template server, how to manually install the Node.js programming environment, and how the configured instance was turned into a reusable custom AMI.

It also details how the AMI was validated by creating a second EC2 instance from the modified image and running “Node.js” commands to verify that the software installation was retained in the AMI. Relevant screenshots are attached, along with technical details to aid in replicating the deployment procedure in the AWS environment.

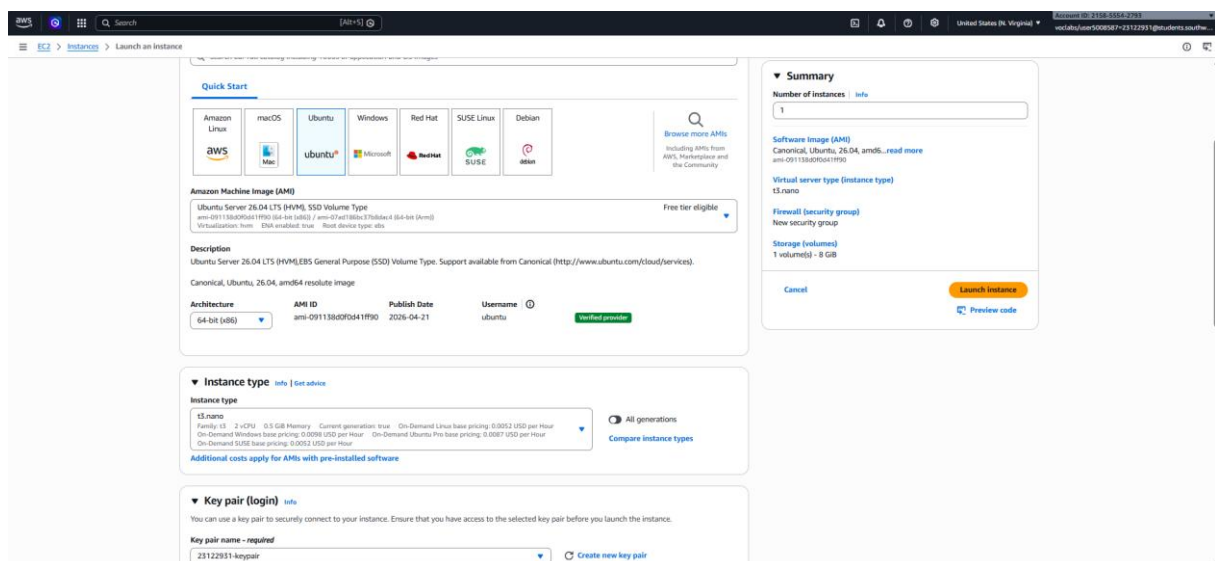
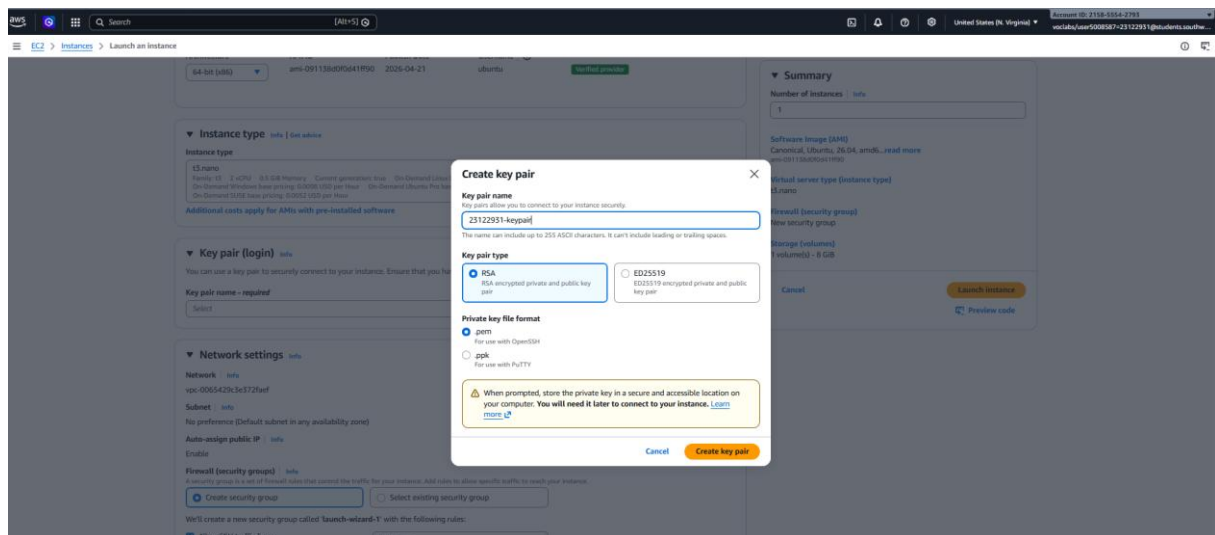
Step 1: The AWS search menu was used to locate and open the EC2 service in the AWS Management Console. The EC2 dashboard was opened successfully, displaying the available EC2 resources and the Launch Instance option.



Step 2: The instance was named 23122931-ec2-template-instance. The latest Ubuntu Linux AMI was selected with a 64-bit specification, and the instance type was set to “t3.nano”.



Step 3: A new RSA key pair named 23122931-keypair was created and configured in “.pem” format to enable secure SSH access to the EC2 instance. The created key pair was successfully attached to the EC2 instance configuration.



Step 4: The network security group and storage settings were configured for the EC2 instance. SSH traffic was enabled to support secure remote access, and the storage volume was configured with 12 GiB of gp3 storage, as required in the brief.

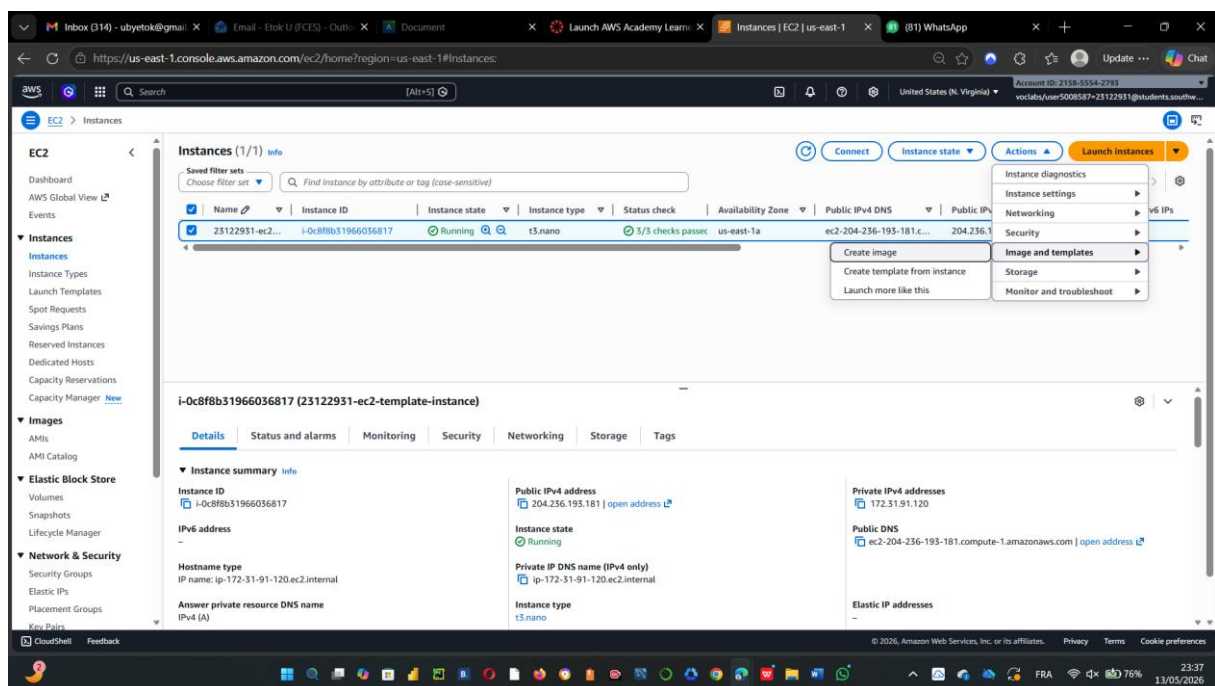
The screenshot shows the AWS 'Launch instance' wizard. The 'Configure storage' step is selected, showing a root volume of 12 GB gp3. The 'Summary' panel on the right displays the instance configuration: t3.nano, Canonical, Ubuntu 20.04, and a new security group. The 'Launch instance' button is visible in the bottom right corner of the summary panel.

Step 5: The EC2 template instance was launched successfully, and the AWS launch log confirmed that the instance initialization, security groups, and security group rules were created successfully.

The screenshot shows the AWS 'Launch instance' success page. A green banner indicates 'Successfully initiated launch of instance'. Below, the 'Launch log' shows four successful steps: Initializing requests, Creating security groups, Creating security group rules, and Launch initiation. The 'Next Steps' section offers various actions like 'Create billing usage alerts', 'Connect to your instance', 'Connect an RDS database', 'Create EBS snapshot policy', 'Manage detailed monitoring', and 'Create Load Balancer'.

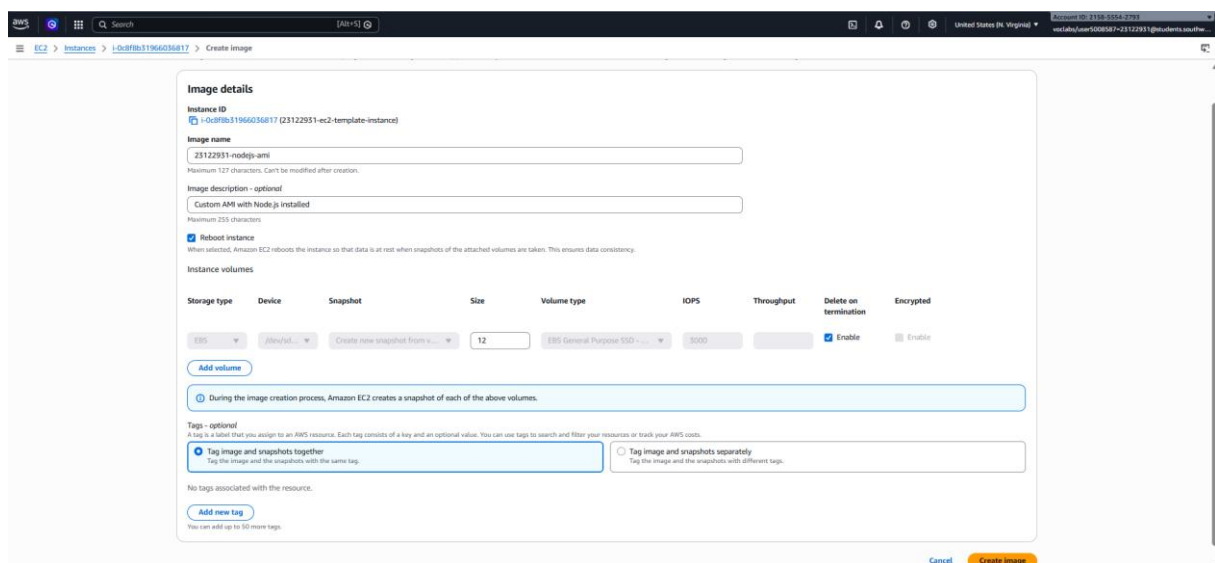
Step 6: The AWS EC2 Dashboard was opened, and the previously created template instance named “23122931-ec2-template-instance” was selected from the list of running instances. To begin creating a reusable Amazon Machine Image, the following navigation path was used from the EC2 Instances page: Actions → Image and templates → Create image.

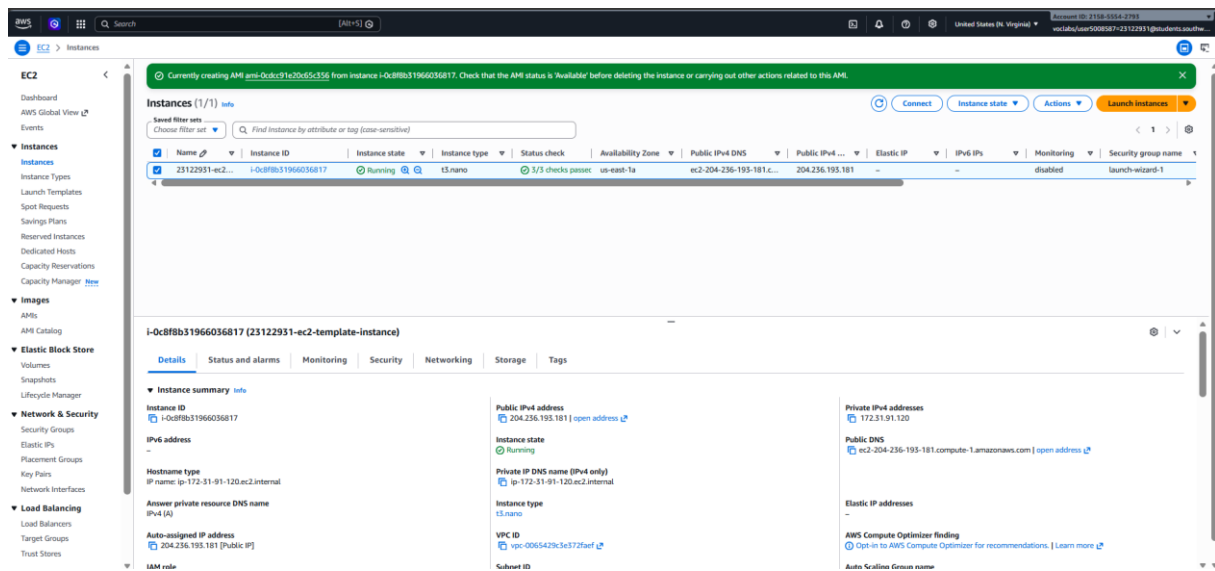
This process created a custom AMI with pre-installed Node.js, which was later used to launch and validate a new EC2 instance.



Step 7: The AMI configuration page was completed to create a reusable machine image from the running EC2 template instance. The image name was set to 23122931-nodejs-ami, and the image description was updated to indicate that the AMI included pre-installed Node.js.

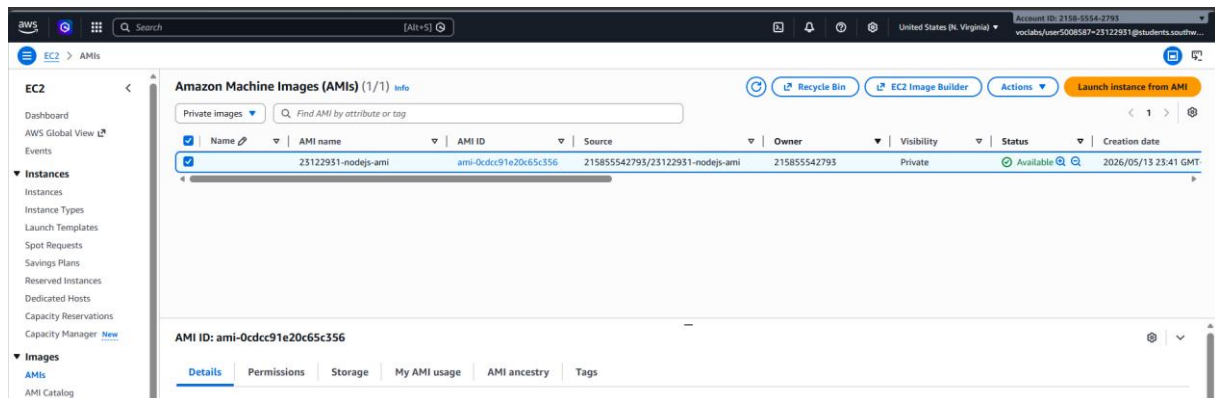
The Reboot instance option remained enabled to ensure consistency during snapshot creation; the storage configuration was set to 12 GB of GP3 storage; and the option to “tag image and snapshots together” was selected. After confirming the configuration settings, the Create image button was selected to generate the custom Amazon Machine Image containing the installed Node.js environment.



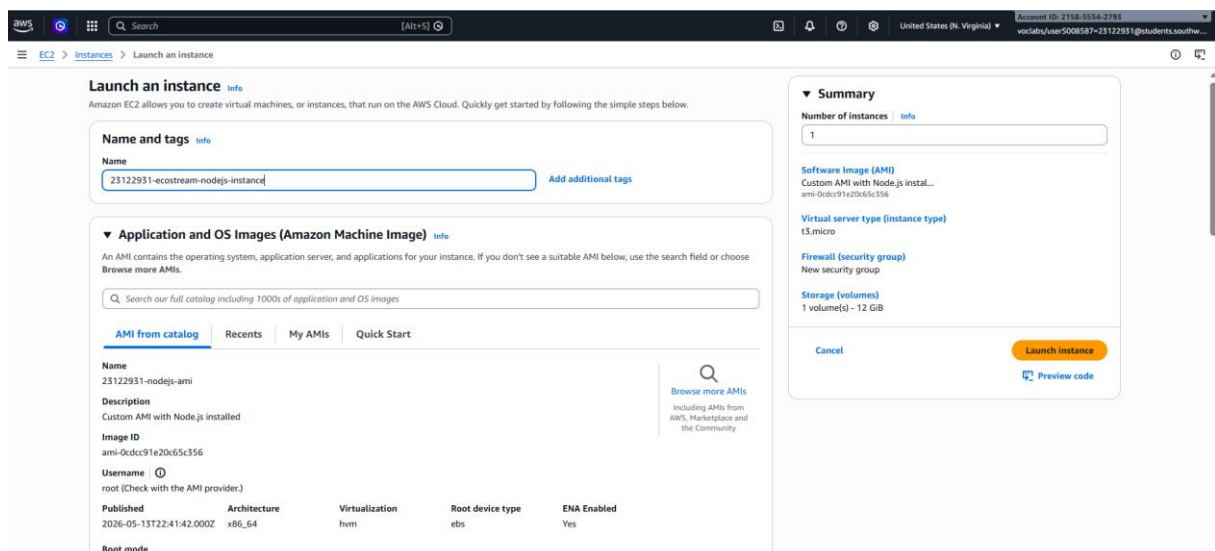


Step 8: From the EC2 navigation panel on the left side of the AWS console, the Images section was expanded, and AMIs were selected to open the Amazon Machine Images page. The Private images category was then used to display the reusable custom AMIs created within the account.

The reusable Node.js AMI named “23122931-nodejs-ami” was selected from the list, and its status was confirmed as Available, indicating that the custom AMI containing the installed Node.js environment had been successfully created and was ready to be used to launch a new EC2 validation instance.



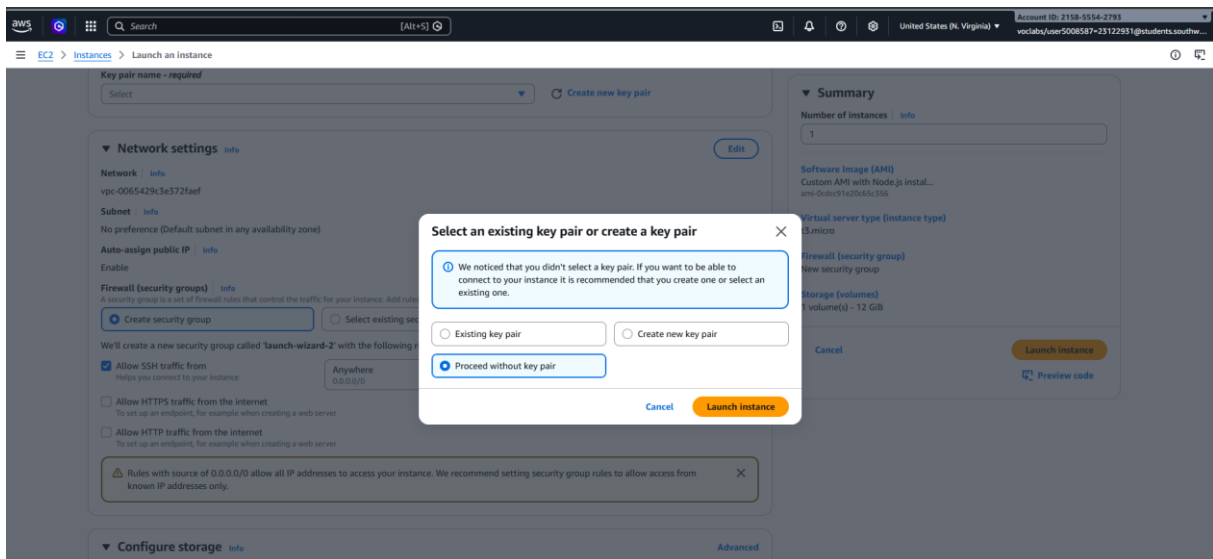
Step 9: On the launch configuration page, the instance name was set to “23122931-ecostream-nodejs-instance,” as required in the assessment brief. The previously created reusable AMI containing Node.js was automatically attached as the software image for the new EC2 instance.



Step 10: While configuring the validation EC2 instance, the Network settings section was reviewed, and a new security group was selected within the firewall configuration to control

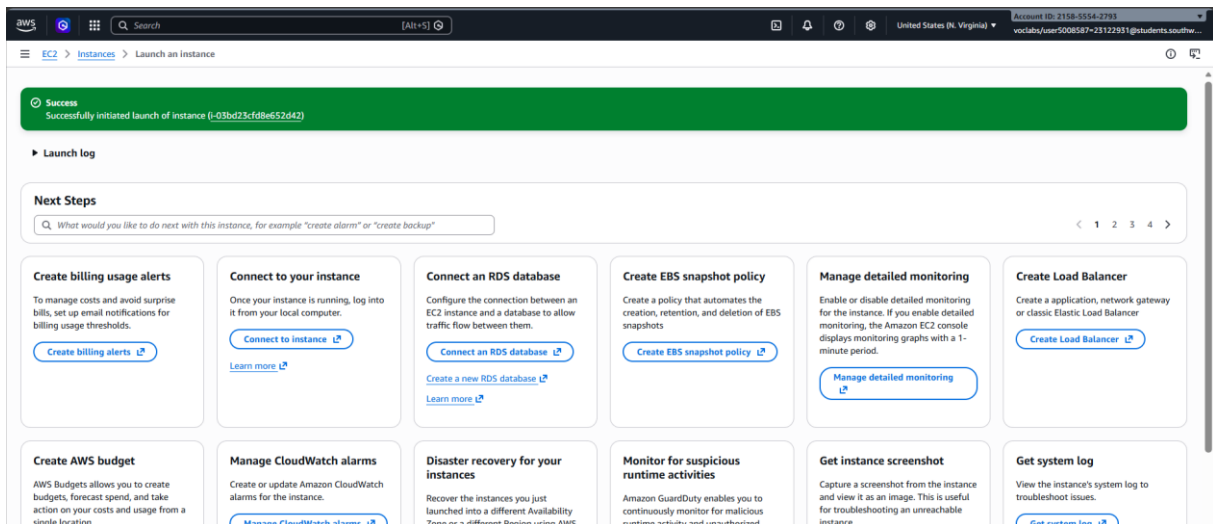
inbound access to the instance. During the launch process, AWS displayed a prompt requesting a key pair selection before the instance could be created. Since validation was performed via the AWS browser-based EC2 Instance Connect terminal, the “Proceed without key pair” option was selected.

The “Allow SSH traffic from” option was selected, and the remaining launch settings were retained at their default values to ensure the validation instance launched successfully using the reusable Node.js AMI.



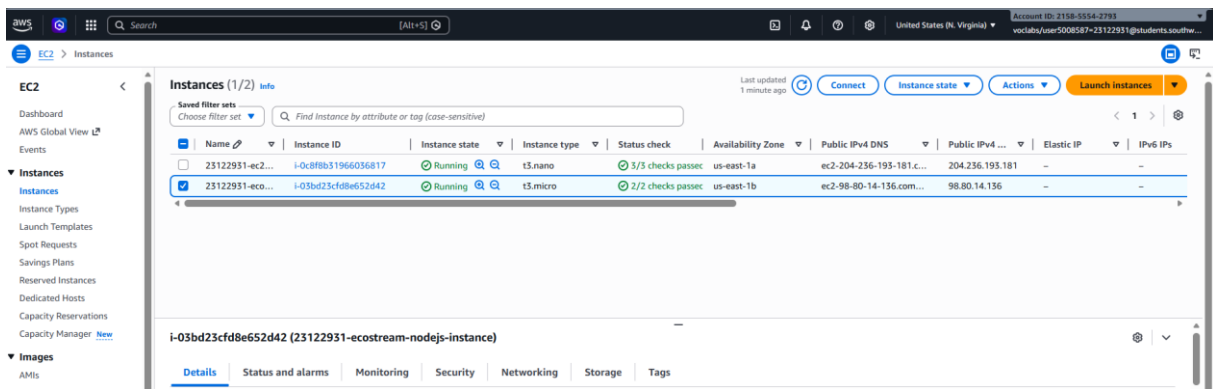
Step 11: After completing the launch configuration, the Launch instance button was selected to deploy the new EC2 validation instance from the reusable Node.js AMI.

The AWS console displayed a successful launch confirmation message indicating that the EC2 validation instance had been created successfully, making the validation environment ready for testing the installed Node.js software.



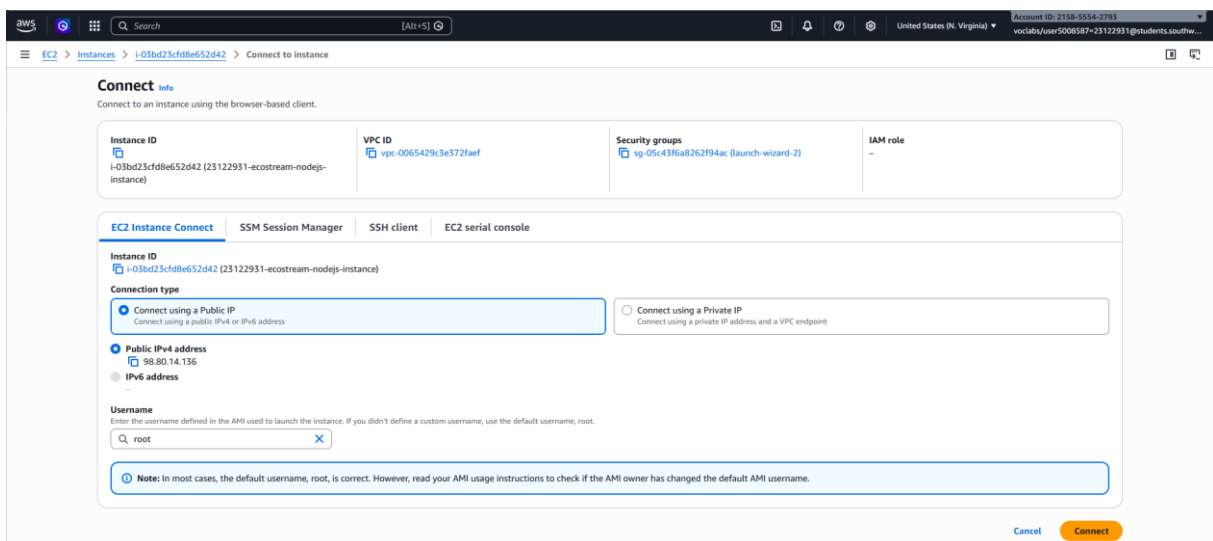
Step 12: After launching the validation instance, the EC2 navigation panel was used to return to the Instances section to verify the status of both EC2 instances. The original template instance “23122931-ec2-template-instance” and the newly launched validation instance “23122931-ecostream-nodejs-instance” were both displayed as Running instances. The status checks also confirmed that the instances were operating correctly, indicating that the validation instance created from the reusable Node.js AMI had launched successfully and was ready for terminal-based Node.js validation.

The newly created validation instance, 23122931-ecostream-nodejs-instance, was selected from the Instances list for software validation as required in the brief, and the connect button was used to open the instance connection page.



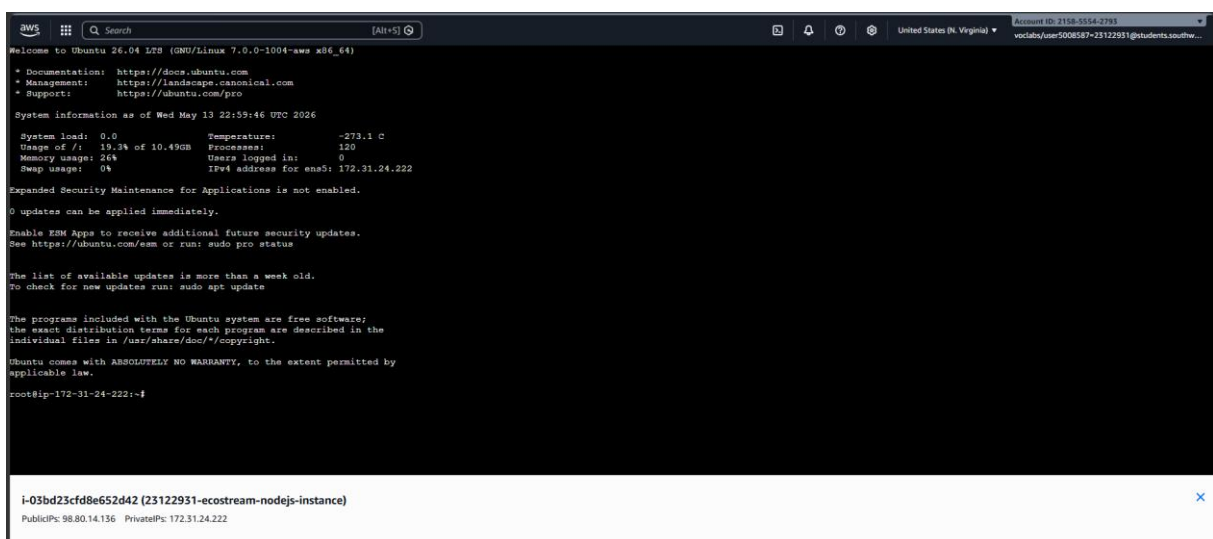
Step 13: On the connection page, the EC2 Instance Connect tab was selected to establish a browser-based terminal connection to the validation instance. The “Connect using a Public IP” option remained selected to allow remote browser access to the EC2 instance, while the default username root was retained because it matched the configuration of the reusable “Node.js” AMI.

These settings were confirmed before selecting the “Connect” button to open the terminal environment required for validating the installed “Node.js” software.



Step 14: After selecting the “Connect” button from the EC2 Instance Connect page, a browser-based terminal session was successfully opened for the validation instance “23122931-ecostream-nodejs-instance”.

The terminal displayed Ubuntu operating system information, system status details, and the root command prompt, confirming that the EC2 validation instance created from the reusable “Node.js” AMI was running successfully and accessible via the AWS browser terminal. This terminal session was required to perform the “Node.js” validation commands specified in the assessment brief.



Step 15: This command, “sudo apt update,” was used to refresh the package index and retrieve the latest available information about software packages from the Ubuntu repositories before proceeding with the Node.js validation process.

The terminal output confirmed that the system successfully connected to the Ubuntu repositories and downloaded updated package information required for package verification and software management within the EC2 validation instance.

```
aws
[Alt+S]
Account ID: 2158-5554-2793
voclabs/user5008587=23122931@students.southw...

Expanded Security Maintenance for Applications is not enabled.
0 updates can be applied immediately.
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Wed May 13 22:59:48 2026 from 18.206.107.27
root@ip-172-31-24-222:~# sudo apt update
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease [136 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease [136 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease [136 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security InRelease [136 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 Packages [1483 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main Translation-en [524 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64 Components [395 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64 c-n-f Metadata [32.4 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 Packages [16.3 MB]
Get:10 http://security.ubuntu.com/ubuntu resolute-security/main amd64v3 Packages [40.4 kB]
Get:11 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [18.1 kB]
Get:12 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [2848 B]
Get:13 http://security.ubuntu.com/ubuntu resolute-security/main amd64 c-n-f Metadata [968 B]
Get:14 http://security.ubuntu.com/ubuntu resolute-security/universe amd64v3 Packages [39.9 kB]
Get:15 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [12.1 kB]
Get:16 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [20.6 kB]

i-03bd23cfd8e652d42 (23122931-ecostream-nodejs-instance)
PublicIPs: 98.80.14.136 PrivateIPs: 172.31.24.222
```

Step 16: The command “sudo apt install nodejs -y” was used to install Node.js and its dependencies on the EC2 validation instance. The “-y” option automatically approved the installation process without requiring manual confirmation from the user.

The terminal output confirmed that the system successfully downloaded and installed the Node.js package, along with its supporting libraries and dependencies, indicating that the Node.js environment was properly configured on the EC2 validation instance.

```
aws
[Alt+S]
Account ID: 2158-5554-2793
voclabs/user5008587=23122931@students.southw...

Get:48 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports/restricted amd64 c-n-f Metadata [120 B]
Get:49 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports/multiverse amd64 Components [216 B]
Get:50 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports/multiverse amd64 c-n-f Metadata [120 B]
Fetched 31.5 MB in 4s (8064 kB/s)
26 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@ip-172-31-24-222:~# sudo apt install nodejs -y
Installing:
nodejs

Installing dependencies:
libada-url0-3 libliltip9.3 libsimdjson29 node-acorn node-brace-expansion node-corepack node-llhttp node-minimatch node-semver node-xtend
libcblas2 libnode127 libsimdutf31 node-balanced-match node-cjs-module-lexer node-debug node-lru-cache node-ma node-undici nodejs-doc

Suggested packages:
npm

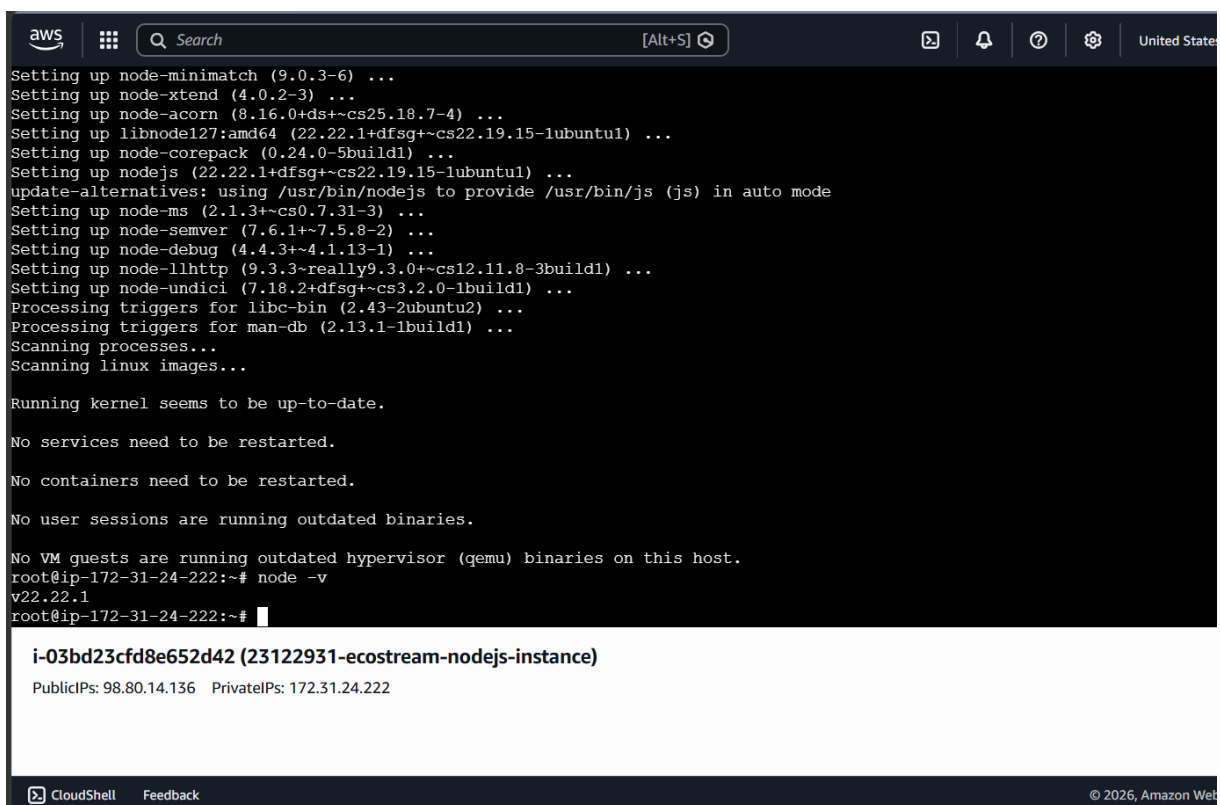
Summary:
Upgrading: 0, Installing: 21, Removing: 0, Not Upgrading: 26
Download size: 25.8 MB
Space needed: 106 MB / 8839 MB available

Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libsimdutf31 amd64 8.0.0-1 [105 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 libada-url0-3 amd64 3.4.3-1 [244 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libcblas2 amd64 1.34.6-1 [102 kB]
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 libliltip9.3 amd64 9.3.3-really9.3.0+~cs12.11.8-3build1 [30.5 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 libsimdjson29 amd64 4.2.4-1 [85.1 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 node-corepack all 0.24.0-5build1 [761 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 nodejs amd64 22.22.1+dfsg~cs22.19.15-1ubuntu1 [367 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/universe amd64v3 node-xtend all 4.0.2-3 [3902 B]

i-03bd23cfd8e652d42 (23122931-ecostream-nodejs-instance)
PublicIPs: 98.80.14.136 PrivateIPs: 172.31.24.222
```

Step 17: The command “node -v” was used to verify that Node.js had been successfully installed on the validation instance.

The terminal output displayed the installed Node.js version “v22.22.1”, confirming that the Node.js software package was properly installed and operational within the EC2 validation instance created from the reusable custom AMI.



```
aws [Search] [Alt+S]
Setting up node-minimatch (9.0.3-6) ...
Setting up node-xtend (4.0.2-3) ...
Setting up node-acorn (8.16.0+ds+~cs25.18.7-4) ...
Setting up libnode127:amd64 (22.22.1+dfsg+~cs22.19.15-1ubuntu1) ...
Setting up node-corepack (0.24.0-5build1) ...
Setting up nodejs (22.22.1+dfsg+~cs22.19.15-1ubuntu1) ...
update-alternatives: using /usr/bin/nodejs to provide /usr/bin/js (js) in auto mode
Setting up node-ms (2.1.3+~cs0.7.31-3) ...
Setting up node-semver (7.6.1+~7.5.8-2) ...
Setting up node-debug (4.4.3+~4.1.13-1) ...
Setting up node-llhttp (9.3.3~really9.3.0+~cs12.11.8-3build1) ...
Setting up node-undici (7.18.2+dfsg+~cs3.2.0-1build1) ...
Processing triggers for libc-bin (2.43-2ubuntu2) ...
Processing triggers for man-db (2.13.1-1build1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

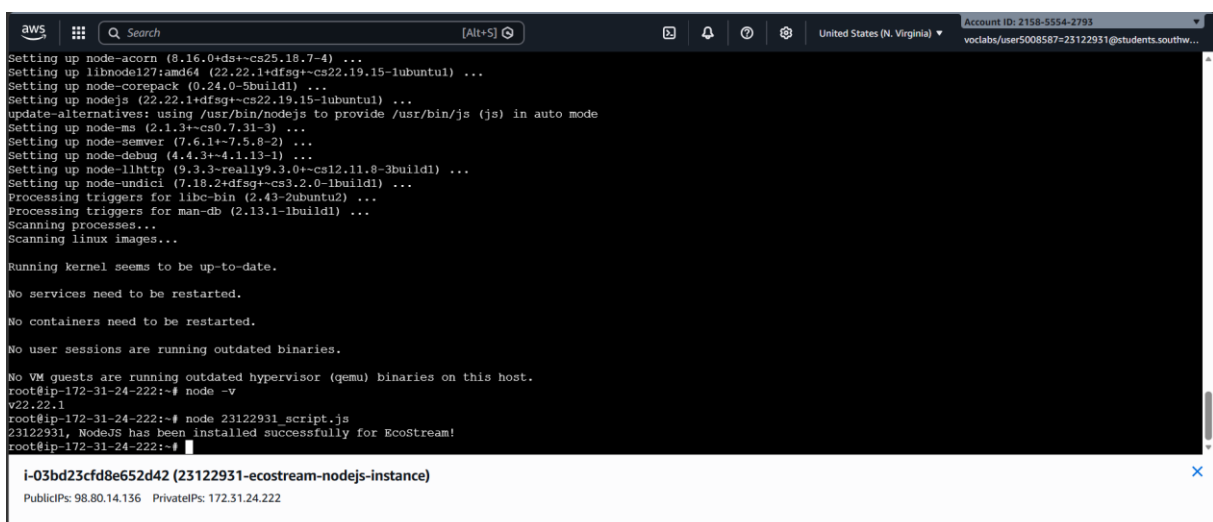
No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@ip-172-31-24-222:~# node -v
v22.22.1
root@ip-172-31-24-222:~#
```

i-03bd23cfd8e652d42 (23122931-ecostream-nodejs-instance)
PublicIPs: 98.80.14.136 PrivateIPs: 172.31.24.222

CloudShell Feedback © 2026, Amazon Web

Step 18: The command “node 23122931_script.js” was used to run the Java script validation file created during the AMI testing process.

The terminal output displayed the message: “23122931, NodeJS has been installed successfully for EcoStream!” confirming that the Node.js environment was functioning correctly within the EC2 validation instance and that the reusable custom AMI had been successfully validated.



```
aws [Search] [Alt+S]
Setting up node-acorn (8.16.0+ds+~cs25.18.7-4) ...
Setting up libnode127:amd64 (22.22.1+dfsg+~cs22.19.15-1ubuntu1) ...
Setting up node-corepack (0.24.0-5build1) ...
Setting up nodejs (22.22.1+dfsg+~cs22.19.15-1ubuntu1) ...
update-alternatives: using /usr/bin/nodejs to provide /usr/bin/js (js) in auto mode
Setting up node-ms (2.1.3+~cs0.7.31-3) ...
Setting up node-semver (7.6.1+~7.5.8-2) ...
Setting up node-debug (4.4.3+~4.1.13-1) ...
Setting up node-llhttp (9.3.3~really9.3.0+~cs12.11.8-3build1) ...
Setting up node-undici (7.18.2+dfsg+~cs3.2.0-1build1) ...
Processing triggers for libc-bin (2.43-2ubuntu2) ...
Processing triggers for man-db (2.13.1-1build1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

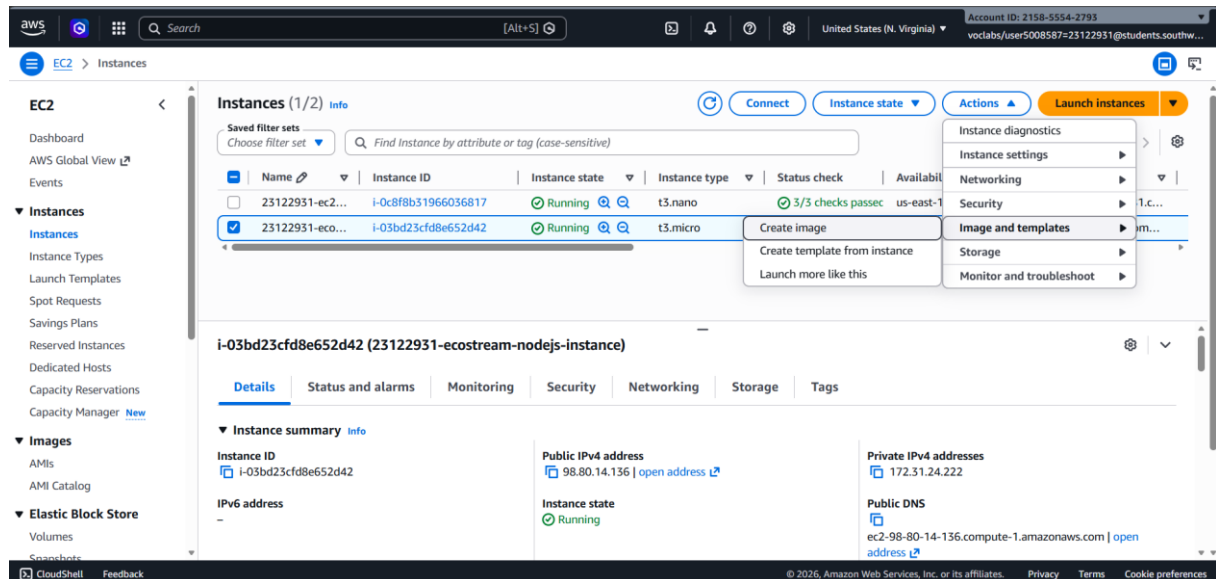
No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
root@ip-172-31-24-222:~# node -v
v22.22.1
root@ip-172-31-24-222:~# node 23122931_script.js
23122931, NodeJS has been installed successfully for EcoStream!
root@ip-172-31-24-222:~#
```

i-03bd23cfd8e652d42 (23122931-ecostream-nodejs-instance)
PublicIPs: 98.80.14.136 PrivateIPs: 172.31.24.222

Step 19: After successfully validating Node.js within the EC2 validation instance, the EC2 navigation panel was used to return to the Instances section. The validation instance “23122931-ecostream-nodejs-instance” was selected, and the following navigation path was used to begin creating an additional backup AMI: Actions → Image and templates → Create image

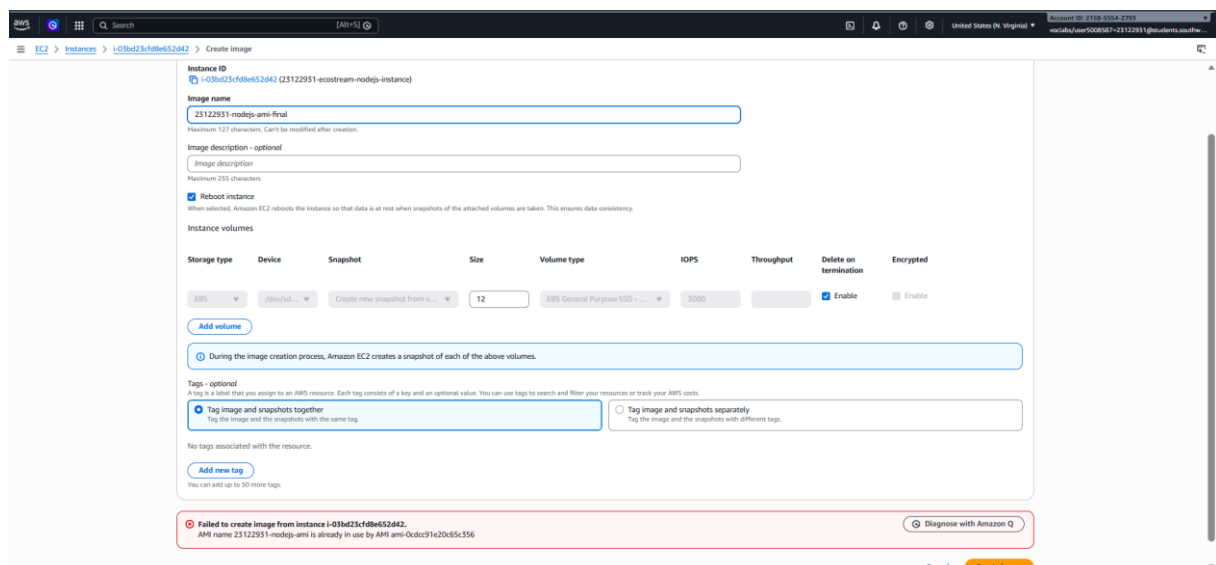
This step was performed to generate a reusable backup image from the fully validated Node.js EC2 instance.



Step 20: The Create image configuration page was opened for the validation instance after navigating through Actions → Image and templates → Create image. Initially, the AMI name “23122931-nodejs-ami” was entered; however, AWS displayed an error message indicating that the name was already associated with an existing AMI that had been previously created successfully.

To resolve this issue, the AMI name was changed to “23122931-nodejs-ami-final”. The “Reboot instance” option remained selected to ensure AWS creates a consistent snapshot of the instance during image creation. Under the storage configuration, the default 12 GB storage volume settings were retained, while the “Delete on termination” option remained enabled. The “Tag image and snapshots together” option was also selected so that both the AMI and its associated snapshots would share the same tagging configuration.

After confirming and retaining the remaining default snapshot and storage settings, the “Create image” button was selected to proceed with AMI creation.

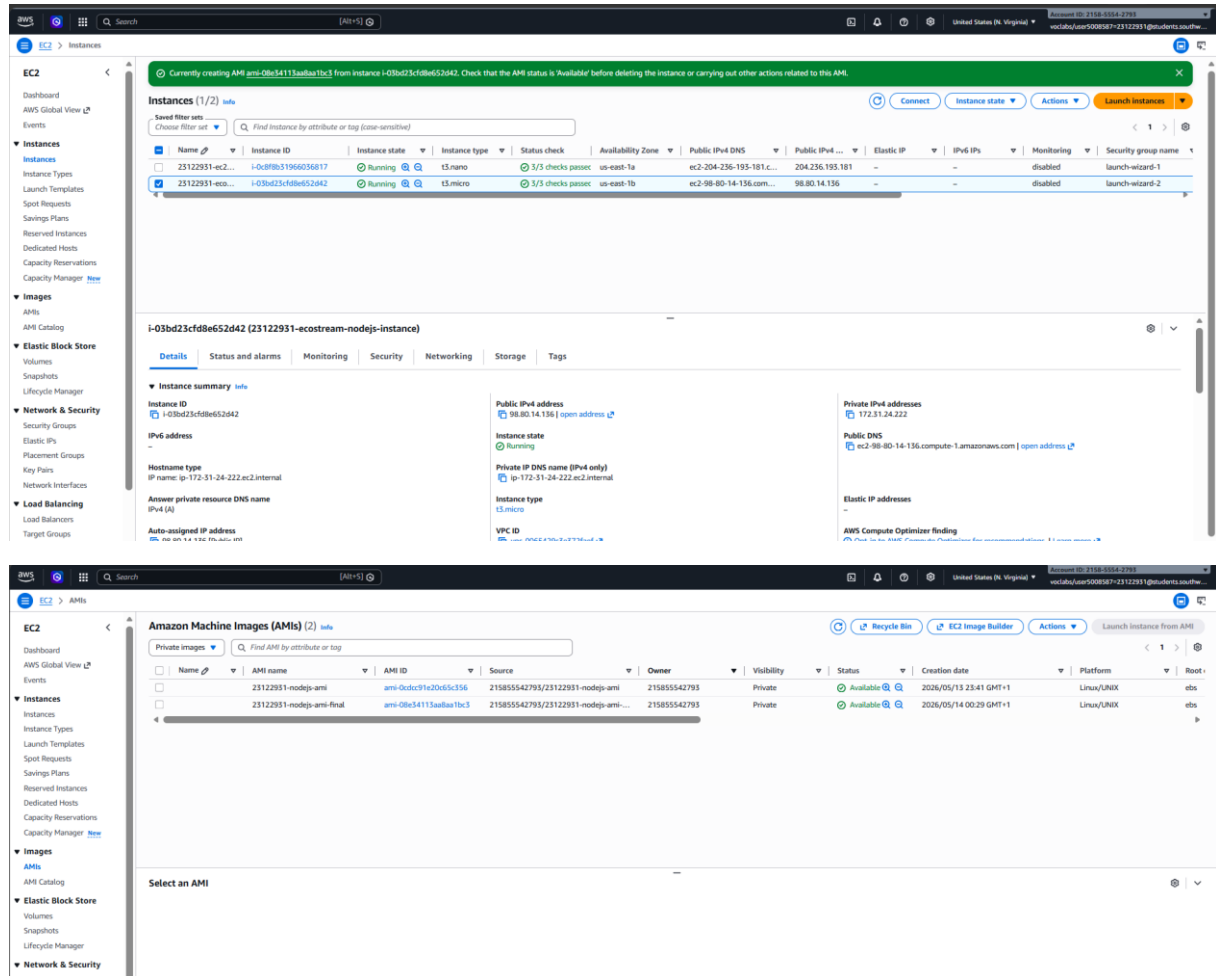


Step 21: After the updated AMI creation request was submitted, a green status notification appeared at the top of the page confirming that AWS was currently creating the AMI from the validation instance “23122931-ecostream-nodejs-instance”.

Both the original template instance “23122931-ec2-template-instance” and the validation instance “23122931-ecostream-nodejs-instance” remained in a Running state with all status checks passed successfully during the AMI creation process. This confirmed that the reusable “Node.js” AMI creation had started successfully and that the instance was operating correctly while AWS generated the machine image snapshots in the background.

The EC2 navigation panel was used to navigate to Images → AMIs to verify the successful creation of the reusable Node.js machine images. Within the Private images section, both custom AMIs named “23122931-nodejs-ami” and “23122931-nodejs-ami-final” were displayed in the AMIs list.

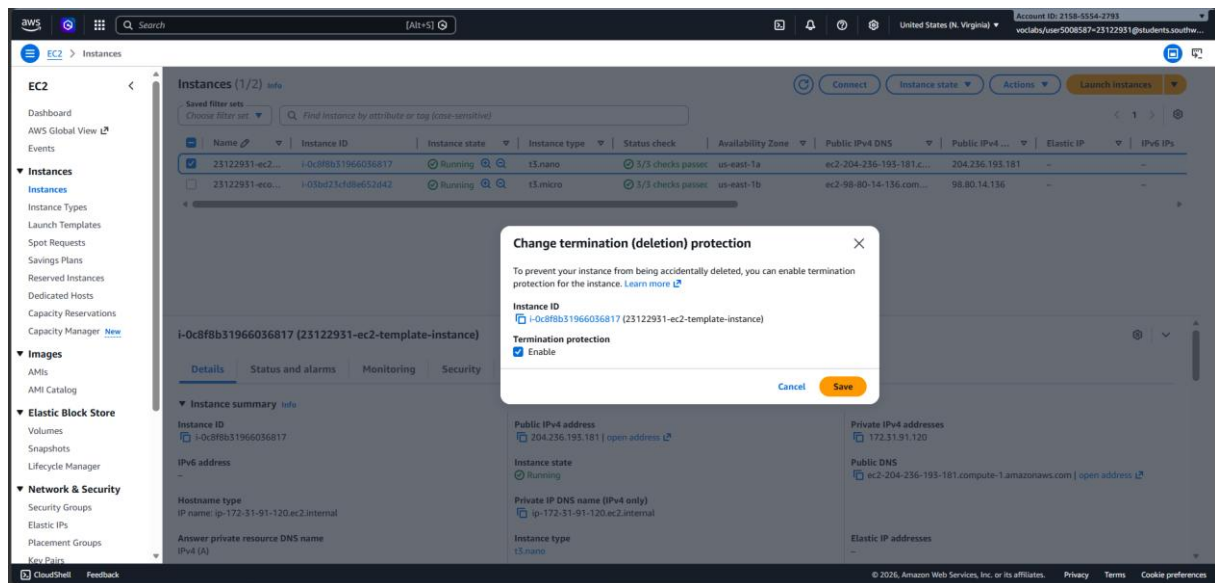
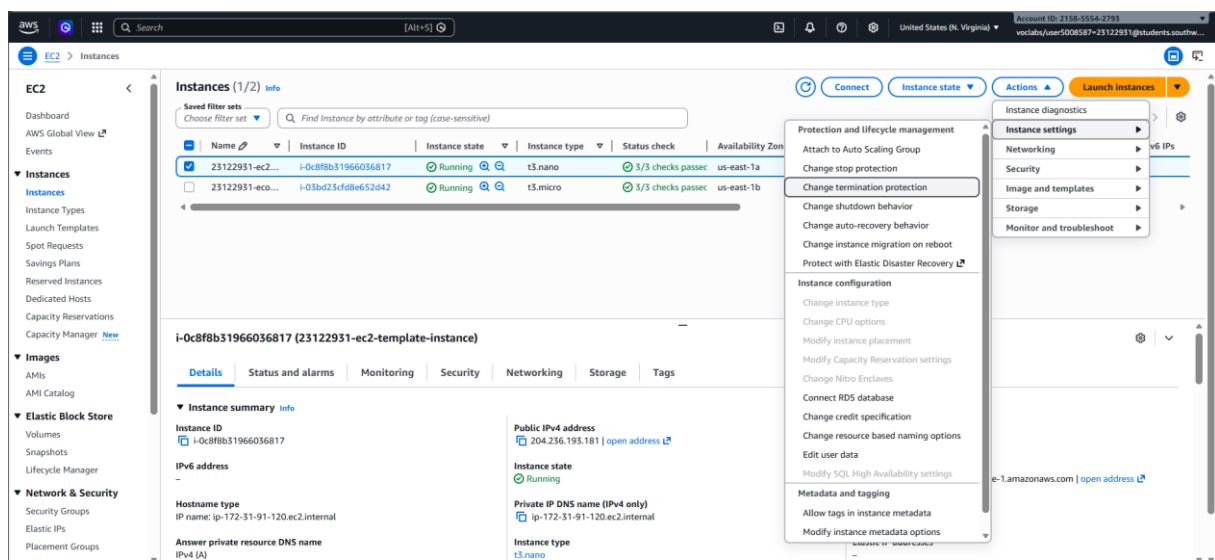
The status of both AMIs was shown as Available, confirming that AWS had successfully completed AMI creation and snapshot generation and was ready to launch EC2 validation instances with “Node.js” pre-installed.



Step 22: The EC2 navigation panel was used to remain in the Instances section, where the original template instance, “23122931-ec2-template-instance,” was selected. The following navigation path was then used to configure deletion protection for the template instance:

Actions → Instance settings → Change termination protection

This option was selected to prevent the EC2 template instance from being accidentally deleted during future AWS operations. Enabling “termination protection” ensured that the template instance complied with the assessment requirement for protecting critical insurance analytics infrastructure from accidental termination.



Step 23: The validation instance “23122931-ecostream-nodejs-instance” was then selected separately. The following navigation path was again used:

Actions → Instance settings → Change termination protection

This option was selected to prevent the EC2 validation instance from being accidentally deleted during future AWS operations. Enabling “termination protection” ensured that the validation instance complied with the assessment requirement for protecting critical insurance analytics infrastructure from accidental termination.

AWS displayed a green success notification confirming that termination protection had been successfully enabled for the template instance.

Successfully enabled termination protection for instance i-0c8fb31966036817. The instance can't be deleted.

Instances (1/2) Info

Save filter sets
Choose filter set
Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone
23122931-ec2...	i-0c8fb31966036817	Running	t3.nano	3/3 checks passed	us-east-1a
23122931-eco...	i-03bd23cfd8e652d42	Running	t3.micro	3/3 checks passed	us-east-1b

i-03bd23cfd8e652d42 (23122931-ecostream-nodejs-instance)

Details Status and alarms Monitoring Security Networking Storage Tags

Instance summary

Instance ID: i-03bd23cfd8e652d42
Public IPv4 address: 98.80.14.136 | open address
Instance state: Running
Private IP DNS name (IPv4 only): ip-172-31-24-222.ec2.internal
Answer private resource DNS name: IPv4 (A)

Protection and lifecycle management

- Attach to Auto Scaling Group
- Change stop protection
- Change termination protection
- Change shutdown behavior
- Change auto-recovery behavior
- Change instance migration on reboot
- Protect with Elastic Disaster Recovery

Instance configuration

- Change instance type
- Change CPU options
- Modify instance placement
- Modify Capacity Reservation settings
- Change Nitro Enclaves
- Connect RDS database
- Change credit specification
- Change resource based naming options
- Edit user data
- Modify SQL High Availability settings

Metadata and tagging

- Elastic IP addresses

Successfully enabled termination protection for instance i-0c8fb31966036817. The instance can't be deleted.

Instances (1/2) Info

Save filter sets
Choose filter set
Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
23122931-ec2...	i-0c8fb31966036817	Running	t3.nano	3/3 checks passed	us-east-1a	ec2-204-236-193-181...	204.236.193.181	-	-
23122931-eco...	i-03bd23cfd8e652d42	Running	t3.micro	3/3 checks passed	us-east-1b	ec2-98-80-14-136.com...	98.80.14.136	-	-

i-03bd23cfd8e652d42 (23122931-ecostream-nodejs-instance)

Details Status and alarms Monitoring Security Networking Storage Tags

Instance summary

Instance ID: i-03bd23cfd8e652d42
Public IPv4 address: 98.80.14.136 | open address
Instance state: Running
Private IP DNS name (IPv4 only): ip-172-31-24-222.ec2.internal
Answer private resource DNS name: IPv4 (A)

Change termination (deletion) protection

To prevent your instance from being accidentally deleted, you can enable termination protection for the instance. Learn more

Instance ID: i-03bd23cfd8e652d42 (23122931-ecostream-nodejs-instance)

Termination protection: ☒ Enable

Cancel Save

Step 24: AWS displayed a green success notification confirming that termination protection had been successfully enabled for the validation instance.

Successfully enabled termination protection for instance i-03bd23cfd8e652d42. The instance can't be deleted.

Instances (1/2) Info

Save filter sets
Choose filter set
Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
23122931-ec2...	i-0c8fb31966036817	Running	t3.nano	3/3 checks passed	us-east-1a	ec2-204-236-193-181...	204.236.193.181	-	-
23122931-eco...	i-03bd23cfd8e652d42	Running	t3.micro	3/3 checks passed	us-east-1b	ec2-98-80-14-136.com...	98.80.14.136	-	-

i-03bd23cfd8e652d42 (23122931-ecostream-nodejs-instance)

Details Status and alarms Monitoring Security Networking Storage Tags

Instance summary

Instance ID: i-03bd23cfd8e652d42
Public IPv4 address: 98.80.14.136 | open address
Instance state: Running
Private IP DNS name (IPv4 only): ip-172-31-24-222.ec2.internal
Answer private resource DNS name: IPv4 (A)

Private IPv4 addresses

172.31.24.222

Public DNS

ec2-98-80-14-136.compute-1.amazonaws.com | open address

Elastic IP addresses